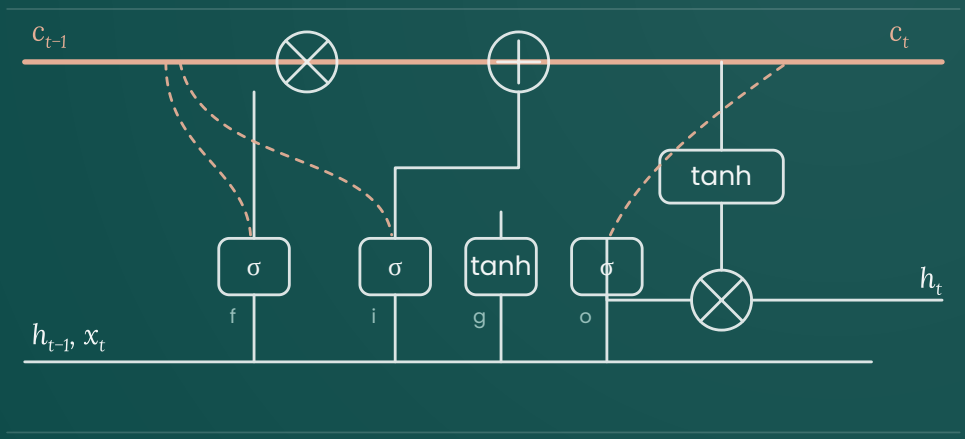


ADVANCED NLP · LAB PROJECT ON RECURRENT NETWORKS

Comparing LSTM cell variants for *drug review* sentiment classification

A vanilla LSTM against Working Memory Connections, the Coupled Input-Forget Gate and a projected LSTM, all built from `nn.Linear`, on 128,458 deduplicated patient reviews.

TEAM

Aya EL MOUDEN
Adam HADOUTI
Moustapha Abdelkerim KOIBORO
Amine LASSRI
Mohamed Hamza EL ASRI

SUPERVISED BY

Pr. Youness MOUKAFIH
Alaa ZNIBER

INSTITUTION

Big Data & AI
ESIN – Université Internationale de Rabat
2026 / 2027

ABSTRACT

We classify Drugs.com patient reviews as negative, neutral or positive and ask whether changing the LSTM cell changes the result. Four cells are implemented from scratch with `nn.Linear`: a vanilla LSTM baseline, the Coupled Input-Forget Gate (CIFG, 25 % fewer recurrent parameters), Working Memory Connections (WMC, 37 % more) and a projected LSTM (LSTMP). After removing 54,474 duplicated reviews and 32,131 reviews that leaked from the training set into the test set, all models are trained on identical splits with the same budget and evaluated with macro-F1. Under the base configuration the cells differ little (mean macro-F1 0.530–0.539 over three seeds), but across nine paired settings the vanilla LSTM is always last and CIFG matches or beats it with 25 % fewer parameters. Training choices matter more: a class-weighted loss adds up to 0.048 macro-F1 by lifting neutral F1 from below 0.05 to 0.25, a higher learning rate up to 0.026 and a forget-gate bias of 1 up to 0.018. The selected model, a class-weighted CIFG-LSTM, reaches **0.583 test macro-F1**.

128 458	8.9 %	4 cells	0.583
reviews kept after deduplication (from 215 063)	neutral reviews: the minority class that drives macro-F1	vanilla, WMC, CIFG, LSTMP, verified against their equations	best test macro-F1 (model selected on validation)

01 Goals of the task
02 Exploratory data analysis
03 Modeling approach
04 Hyperparameter design
05 Performance and evaluation
06 Potential improvements
A Implementation and verification
B Experiment execution (screenshots)

01

Goals of the task

The task is many-to-one sequence classification: a model reads the text of a patient review and predicts its sentiment class derived from the 1–10 rating. Beyond accuracy, the question is architectural. The two non-vanilla cells we chose pull in opposite directions, which turns the comparison into a test of whether complexity pays off:

- **CIFG removes a gate** ($i_t = 1 - f_t$). Can the LSTM be simplified without losing accuracy?
- **WMC adds cell-to-gate connections.** Do gates that can read the memory they control classify better?
- **LSTMP** is added as a fourth model to compare a different way of saving parameters (a projected recurrent state).

Our goals are to (1) implement the four cells exactly as specified, using only `nn.Linear` for linear maps; (2) compare them under one split, one budget and one evaluation protocol, reporting trainable parameters; and (3) measure how learning rate, optimizer and weight initialisation affect each cell. Code: github.com/adamhadouti/rnn-drug-review.

02

Exploratory data analysis

The UCI Drug Review dataset contains 161,297 training and 53,766 test reviews covering 2,768 drugs and 813 conditions. Each review comes with a 1–10 rating; only the text is used as input. We map ratings 1–4 to *negative*, 5–6 to *neutral* and 7–10 to *positive*.

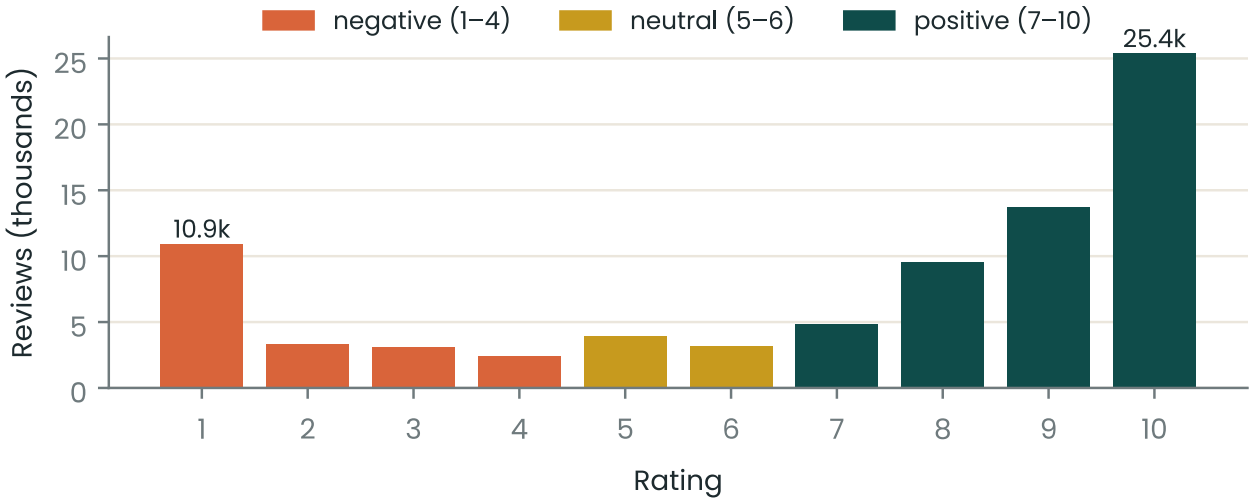


Fig. 1 Training reviews per rating, coloured by the three-class mapping (after deduplication).

Polarised ratings, small neutral class. Ratings pile up at the extremes: 10 is the most frequent score and 1 the third most frequent. The resulting classes are 66.6 % positive, 24.6 % negative and only 8.9 % neutral, with identical shares in the test split (66.0 / 25.1 / 9.0 %). A model that always predicts *positive* would reach about 66 % accuracy with zero neutral recall, so **macro-F1 is our primary metric**.

Duplicates and leakage. The same review is frequently posted under both the brand and the generic name of a drug. We removed exact duplicates inside each split and, most importantly, training reviews that also appear in the test split.

CLEANING STEP	REVIEWS REMOVED	WHY IT MATTERS
Duplicates inside the training split	48 983	over-weights repeated reviews
Duplicates inside the test split	5 491	over-weights repeated reviews in evaluation
Training reviews also present in the test split	32 131	test answers seen during training: inflated scores

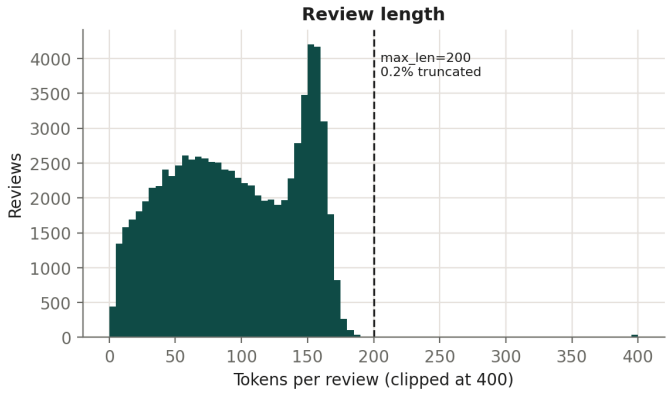


Fig. 2 Review length in tokens; the dashed line is our 200-token truncation limit.

Length. Median 94 tokens, 90th percentile 157, maximum 2,082. The sharp peak at 150–165 tokens is consistent with a length limit on the site's review form, so truncating at 200 tokens affects only **0.2 %** of reviews.

No length shortcut. Mean length is 90, 97 and 96 tokens for negative, neutral and positive reviews: the classes must be told apart by content.

Noise. Reviews contain HTML entities (`'`) and surrounding quotes, which we decode; the `condition` field is truncated in places ("Bipolar Disorde") and is not used.

03

Modeling approach

All models share one classifier and differ only in the recurrent cell. Token ids are embedded (dimension 128, dropout 0.2), read by one recurrent layer (H = 128), and the hidden state at the **last real token** of each review (padding skipped) goes through dropout 0.3 and a linear layer to three logits, trained with cross-entropy.

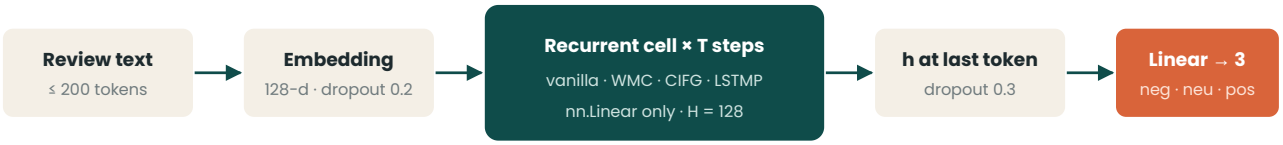


Fig. 3 Shared classification pipeline; only the recurrent cell changes between models.

The four cells

The vanilla cell (equations 1–6 of the assignment) computes input, forget and output gates and a candidate from x_t and h_{t-1}, then updates $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ and $h_t = o_t \odot \tanh(c_t)$. The variants change it as follows (changes in coral):

WMC $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + \mathbf{\tanh(W_{ci}c_{t-1})} + b_i)$, same for f_t (7–8)

$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + \mathbf{\tanh(W_{co}c_t)} + b_o)$ — reads the *new* state (9)

CIFG $\mathbf{i_t = 1 - f_t}$, so $c_t = f_t \odot c_{t-1} + (1 - f_t) \odot g_t$ (14, 17)

LSTMP $m_t = o_t \odot \tanh(c_t)$, $\mathbf{h_t = W_p m_t + b_p}$ with $h_t \in \mathbb{R}^P$, $P = 64$ (24–25)

CELL	RECURRENT PARAMETERS	E = H = 128 (P = 64)	VS. VANILLA	EXPECTED EFFECT
Vanilla	4H(E + H + 1)	131 584	—	baseline
WMC	4H(E + H + 1) + 3H²	180 736	+37 %	better-informed gates
CIFG	3H(E + H + 1)	98 688	-25 %	same accuracy, cheaper
LSTMP	4H(E + P + 1) + PH + P	107 072	-19 %	compressed recurrence

Implementation. Each cell stacks its gate matrices into one `nn.Linear` for x_t and one bias-free `nn.Linear` for h_{t-1}, and computes the input projection for all time steps before the loop. Both are exact rewrites of the equations with one bias per gate. In WMC, c_t is computed before o_t, as equation 9 requires. Nine unit tests check every cell against its equations (Appendix A).

04

Hyperparameter design

Every run uses the same cached splits, budget and evaluation, so a difference between two runs comes only from the setting being studied. The official training split is divided, stratified by class with a fixed seed, into 90 % train and 10 % validation. Because of our GPU budget, models train on a **stratified 50 % subsample** of the train part, which preserves the class balance; validation and test are always complete.

SPLIT	REVIEWS	NEGATIVE	NEUTRAL	POSITIVE
Train (50 % stratified subsample)	36 082	8 877	3 193	24 012
Validation	8 019	1 973	710	5 336
Test (official, deduplicated)	48 275	12 097	4 335	31 843

Base configuration. Lower-cased word tokens; vocabulary built on the training split only (17,706 tokens with frequency ≥ 2, covering 99.6 % of training tokens); length-bucketed batches of 64; Adam with learning rate 10^{-3}; gradient-norm clipping at 1.0; PyTorch default initialisation; at most 5 epochs. The checkpoint with the best **validation** macro-F1 is kept (early stopping, patience 3), and the test set is evaluated once per run.

STUDY	SETTING VARIED (ALL ELSE = BASE)	VALUES	CELLS	RUNS
A · Architecture	cell type, 3 seeds each	vanilla, WMC, CIFG, LSTMP	4	12
B · Learning rate	Adam step size	$3{\cdot}10^{-4}$, $\underline{10^{-3}}$, $3{\cdot}10^{-3}$	vanilla, WMC, CIFG	6
C · Optimizer	update rule	Adam, AdamW, RMSprop (10^{-3}), SGD-Nesterov (0.05)	vanilla, WMC, CIFG	9
D · Initialisation	recurrent weights	default, Xavier, orthogonal, orthogonal + forget bias 1	vanilla, WMC, CIFG	9
G · Class imbalance	loss function	cross-entropy, cross-entropy weighted by inverse class frequency	vanilla, WMC, CIFG	3

Underlined = base value (shared with the seed-1 run of study A). Total: 39 runs on one Colab T4 GPU, about 39 s per epoch for the vanilla cell.

Rationale. The learning rate is usually the most sensitive setting for recurrent networks; SGD is given a larger step, as is standard. Orthogonal recurrent weights preserve gradient norms through time, and a forget-gate bias of 1 makes the cell keep its memory by default early in training (Jozefowicz et al., 2015). Macro-F1 is the selection metric because accuracy rewards ignoring the neutral class.

05

Performance and evaluation

5.1 Architecture comparison

Under the base configuration the four cells are close: mean test macro-F1 ranges from 0.530 (vanilla) to 0.539 (CIFG), a spread similar to CIFG's own seed-to-seed standard deviation (0.008). Three seeds alone therefore cannot rank them. Two practical differences are clear: CIFG matches or beats the baseline with 25 % fewer recurrent parameters and the same speed, while WMC's 37 % extra parameters cost 56 % more time per epoch (61 s vs 39 s), because its output gate must wait for the new cell state. In every run, the neutral class is almost never predicted (neutral F1 ≤ 0.04): macro-F1 is held down by that class, not by the cells.

MODEL	RECURRENT PARAMS	TOTAL PARAMS	TEST ACCURACY	TEST MACRO-F1	NEUTRAL F1	S / EPOCH	SEEDS
Vanilla LSTM	131 584	2 398 339	0.7917 ± 0.0046	**0.5298** ± 0.0033	0.009	39	3
WMC-LSTM	180 736	2 447 491	0.7958 ± 0.0017	**0.5350** ± 0.0037	0.009	61	3
CIFG-LSTM	98 688	2 365 443	0.7950 ± 0.0026	**0.5391** ± 0.0081	0.026	39	3
LSTMP	107 072	2 373 635	0.7936 ± 0.0006	**0.5307** ± 0.0032	0.013	47	3

Mean ± standard deviation over seeds. Same splits, same budget (≤ 5 epochs, 50 % of the training split), checkpoint selected on validation macro-F1.

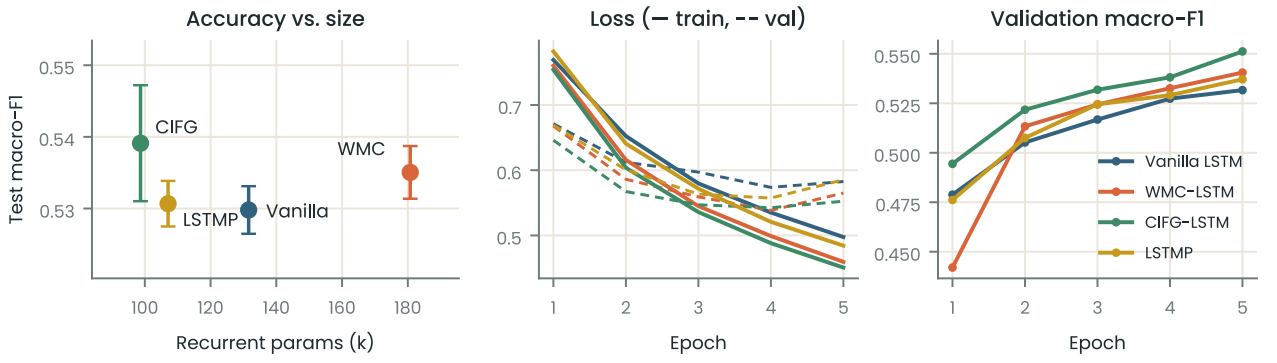


Fig. 4 Left: test macro-F1 (mean ± sd over 3 seeds) against recurrent parameters. Middle and right: learning curves of seed 1; validation loss starts rising after epoch 3 while validation macro-F1 still improves.

5.2 Hyperparameter study

Learning rate is the setting that matters most. Raising it from 10^{-3} to $3{\cdot}10^{-3}$ improves every cell by 0.020 to 0.026 macro-F1, and lowering it to $3{\cdot}10^{-4}$ costs 0.028 to 0.039: within a 5-epoch budget, models that learn faster end better, and the gain comes mostly from the neutral class (neutral F1 up to 0.12). **Adaptive optimizers are interchangeable**: Adam, AdamW and RMSprop stay within 0.005 of each other, while SGD with Nesterov momentum is the worst choice for every cell (−0.030 to −0.039). **For initialisation, the forget-gate bias is what helps**: orthogonal weights alone or Xavier change little, but orthogonal + forget bias 1 is the best scheme for all three cells (+0.009 vanilla, +0.018 WMC, +0.017 CIFG). Across these nine settings, which are the same for all three cells, the ordering of the cells is remarkably stable: the vanilla LSTM is never the best and is last in all nine; CIFG is first in six and WMC in the other three. This paired comparison is stronger evidence than the seed spread of Section 5.1 alone: CIFG's simplification costs nothing here, and WMC's extra memory connections give a small but consistent edge over the baseline.

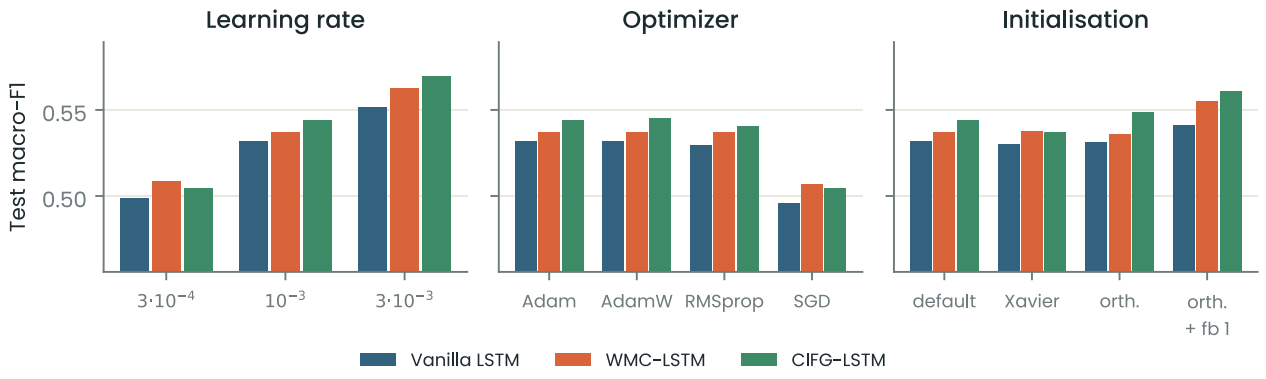


Fig. 5 Test macro-F1 when one setting changes from the base configuration (base = Adam, 10^{-3}, default init).

5.3 Class-weighted loss

The class-weighted loss is the largest single improvement in the study. Weighting cross-entropy by inverse class frequency (neutral reviews count about 7.5 times more than positive ones) raises test macro-F1 by +0.035 (vanilla), +0.048 (WMC) and +0.039 (CIFG). The gain comes from the neutral class, whose F1 rises from below 0.05 to 0.20–0.25. The price is accuracy, which drops by 0.06 to 0.09 because the models now predict neutral, sometimes wrongly. The gated variants benefit more than the vanilla cell: with a loss that forces them to model the minority class, WMC and CIFG both pass 0.58.

MODEL (SEED 1)	TEST ACCURACY	NEUTRAL F1	TEST MACRO-F1	Δ MACRO-F1
Vanilla LSTM	0.7924 → 0.7362	0.022 → **0.196**	0.5319 → **0.5666**	+0.0347
WMC-LSTM	0.7978 → 0.7151	0.015 → **0.254**	0.5370 → **0.5854**	+0.0483
CIFG-LSTM	0.7964 → 0.7082	0.041 → **0.252**	0.5440 → **0.5828**	+0.0388

Unweighted base run → same run with cross-entropy weighted by inverse class frequency; everything else identical.

5.4 Final model

Selected on validation macro-F1 among all 39 runs: **CIFG-LSTM** (adam, lr 0.001, default init, class-weighted loss, seed 1).

Validation macro-F1	0.5939
Test accuracy	0.7082
Test macro-F1	**0.5828**
F1 negative / neutral / positive	0.666 / 0.252 / 0.831
Best epoch · recurrent params	4 · 98 688

The model selected on validation is the **class-weighted CIFG-LSTM**: 0.583 test macro-F1, +0.053 over the vanilla baseline

(three-seed mean of the base configuration), with 25 % fewer recurrent parameters. It now recognises 40 % of neutral reviews, against 8 % for the best unweighted model. Its remaining errors are mostly between neighbouring classes: 16 % of positive reviews and 24 % of negative reviews are predicted neutral, which is expected for reviews rated 4 or 7 that sit next to the neutral band. The class-weighted WMC is marginally better on test (0.585), but it was not the one selected on validation, and its gap to CIFG (0.003) is below the seed variability measured in Section 5.1.

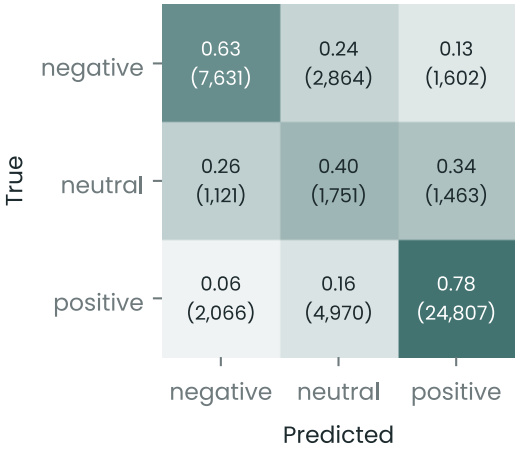


Fig. 6 Normalised confusion matrix on the test set (counts in brackets).

WHAT THE EXPERIMENTS SHOW

1. **The training setup matters more than the cell.** The class-weighted loss (up to +0.048), the learning rate (up to +0.026) and a forget-gate bias of 1 (up to +0.018) each move macro-F1 more than switching cells (about 0.01 under the base configuration). 2. **CIFG is the best trade-off**: the most accurate cell in most settings and the selected final model, with 25 % fewer recurrent parameters and the speed of the vanilla LSTM. 3. **WMC is consistently, slightly better than vanilla**, but 56 % slower per epoch. 4. **The neutral class is the real bottleneck**: without weighting, every model ignores it; with weighting, neutral F1 reaches 0.25, still far below the two polar classes.

06

Potential improvements

1. **Full budget and more seeds.** Train on the whole training split for 10+ epochs and repeat every study over 3 seeds; single-seed differences in studies B–D smaller than the seed spread of study A are not conclusive.
2. **Parameter-matched comparison.** Compare cells at equal size (CIFG with H = 155 and WMC with H = 105 match vanilla's 131k recurrent parameters) to separate the effect of the architecture from that of capacity.
3. **Combine what helped, then go beyond weighting.** Class weighting, learning rate $3{\cdot}10^{-3}$ and forget bias 1 were tested one at a time; combining them is the obvious next run. For the neutral class, focal loss or tuning the decision thresholds on validation could keep the neutral gain at a smaller cost in accuracy; predicting the 1–10 rating with ordinal regression would make confusing 6 with 7 cost less than confusing 1 with 10.
4. **Richer encoders.** Pretrained embeddings (GloVe, fastText), a bidirectional layer, and attention pooling instead of the last hidden state.

Conclusion

We implemented the vanilla, WMC, CIFG and LSTMP cells from `nn.Linear`, verified them against their equations, and compared them on a deduplicated, leakage-free version of the Drug Review dataset under one protocol. The cells are close, but consistently ordered: removing the input gate (CIFG) costs nothing and saves 25 % of the recurrent parameters, while adding memory connections to the gates (WMC) gives a small gain at a 56 % time cost. The larger levers are in training, above all a loss that does not let the model ignore the neutral class. With more compute, the first steps would be to combine the three helpful settings (class weighting, learning rate $3{\cdot}10^{-3}$, forget bias 1), train on the full split for longer, and repeat every comparison over several seeds.

References

1. Landi, F., Baraldi, L., Cornia, M., Cucchiara, R. (2021). Working Memory Connections for LSTM. *Neural Networks*, 144, 334–341.
2. Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R., Schmidhuber, J. (2017). LSTM: A Search Space Odyssey. *IEEE TNNLS*, 28(10), 2222–2232.

3. Sak, H., Senior, A., Beaufays, F. (2014). Long Short-Term Memory Recurrent Neural Network Architectures for Large Scale Acoustic Modeling. *Interspeech*.

4. Jozefowicz, R., Zaremba, W., Sutskever, I. (2015). An Empirical Exploration of Recurrent Network Architectures. *ICML*.

5. Gräßer, F., Kallumadi, S., Malberg, H., Zaunseder, S. (2018). Aspect-Based Sentiment Analysis of Drug Reviews Applying Cross-Domain and Cross-Data Learning. *Digital Health*. Dataset: UCI #462.

APPENDIX A

Implementation and verification

The repository is organised so that every result in this report can be regenerated with three commands. No module uses `nn.LSTM` or `nn.LSTMCell`; PyTorch's `nn.LSTMCell` appears only in the test suite, as an independent reference.

```
rnn-drug-review/
├── src/cells.py          # Vanilla, WMC, CIFG, LSTMP cells (eqs 1–25) + time unroller
├── src/model.py          # embedding → cell → last-token pooling → linear; init schemes
├── src/data.py           # download, HTML cleaning, dedup + leakage removal, fixed splits
├── src/train.py          # one run → results/<name>.json + TensorBoard logs
├── run_experiments.py    # the experiment plan, shardable across team members, resumable
├── src/eda.py · src/aggregate.py   # EDA statistics, result tables and figures
├── tests/test_cells.py   # 9 correctness tests
└── colab_runner.ipynb    # one-click Colab workflow
```

TEST	WHAT IT PROVES
Vanilla vs. `nn.LSTMCell`	with copied weights, identical h_t and c_t (tolerance 10^{-6})
WMC, CIFG, LSTMP vs. equations	outputs recomputed by hand from equations 7–25 match the modules
Parameter counts	each cell's trainable parameters equal the closed-form formulas of Section 3
No built-in LSTM	no `nn.LSTM`, `nn.LSTMCell` or `nn.RNNBase` module in any model
Unroll = step-by-step	the layer's time loop equals calling the cell one step at a time
Padding invariance	a review gets the same logits alone or padded in a batch, for all cells and poolings
Initialisation + gradients	every scheme sets the forget bias correctly and every weight receives a gradient

Reproducing the study. `python -m pytest tests`, then `python run_experiments.py --groups A_arch B_lr C_optim D_init --train_frac 0.5 --epochs 5`, then `python -m src.aggregate`. Finished runs are skipped, so an interrupted Colab session resumes where it stopped, and the plan can be split across team members with `--shard i --num_shards n`.

APPENDIX B

Experiment execution

Screenshots of the Colab runs on a T4 GPU, in execution order.

```
REPO = 'https://github.com/adamhadouti/rnn-drug-review.git'  # <-- edit
SHARD, NUM_SHARDS = 0, 6                                     # <-- edit: your index, team size
!git clone -q $REPO repo
%cd repo
!pip -q install -r requirements.txt

/content/repo

from google.colab import drive
drive.mount('/content/drive')
RESULTS = '/content/drive/MyDrive/rnn_drug_review/results'
!mkdir -p $RESULTS

Mounted at /content/drive

# 1) sanity tests for the cells (seconds)
!python -m pytest tests -q

.........                                                [100%]
9 passed in 4.97s

# 2) EDA figures + dataset statistics (downloads the UCI data on first run)
!python -m src.eda --data_dir data --out figures
...
  "train_raw": 161297,
  "test_raw": 53766,
  "train_dup_removed": 48983,
  "test_dup_removed": 5491,
  "train_test_overlap_removed": 32131
 },
 "train_size": 80183,
 "test_size": 48275,
 "n_drugs": 2768,
 "n_conditions": 813,
```

Fig. 7 Environment set-up: repository cloned, Google Drive mounted for persistent results, the 9 cell tests pass, and the EDA statistics are computed on the full dataset.

```
 "train_raw": 161297,
 "test_raw": 53766,
 "train_dup_removed": 48983,
 "test_dup_removed": 5491,
 "train_test_overlap_removed": 32131,
 "train_size": 72164,
 "train_class_counts": [
  17754,
  6387,
  48023
 ],
 "val_size": 8019,
 "val_class_counts": [
  1973,
  710,
  5336
 ],
 "test_size": 48275,
 "test_class_counts": [
  12097,
  4335,
  31843
 ],
 "vocab_size": 23757,
 "train_token_coverage": 0.9973
}
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792938326.417932    1089 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critica
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792938329.086897    1089 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
[vanilla_h128_L1_adam_lr0.001_bs64_default_s1] device=cuda params={'total': 3172867, 'embedding': 3040896, 'recurrent': 131584, 'classifier': 387}
  ep01 train loss 0.7109 acc 0.7210 | val loss 0.5892 acc 0.7860 F1 0.5161 | 77s
[vanilla_h128_L1_adam_lr0.001_bs64_default_s1] TEST acc 0.7759 macro-F1 0.5089 -> /tmp/timing/vanilla_h128_L1_adam_lr0.001_bs64_default_s1.json
```

Fig. 8 Fixed splits with per-class counts, vocabulary statistics, and a timing epoch on the GPU (`device=cuda`) that set our training budget.

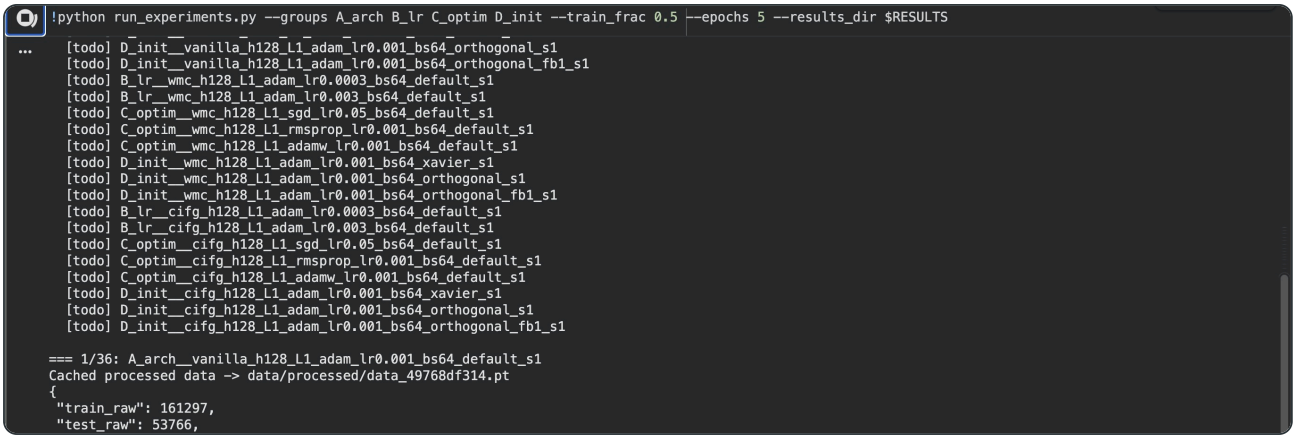


Fig. 9 The 36-run experiment plan (studies A–D) launched with a 50 % stratified training subsample and at most 5 epochs.

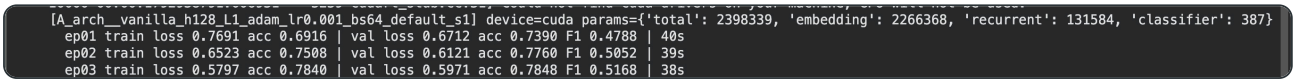


Fig. 10 First run of the plan: parameter breakdown of the vanilla model and per-epoch training and validation metrics (about 39 s per epoch).